

Async JS & APIs

COMPLETE LEARNING GUIDE

What You'll Learn

Almost everything a frontend app does that takes real-world time — fetching data, waiting on a timer, reading a file, waiting for user input — is asynchronous, and JavaScript's single-threaded, non-blocking model handles this through callbacks, Promises, and the `async/await` syntax built on top of them. This submodule traces that evolution: why raw callbacks caused 'callback hell,' how Promises fixed composability with `.then().catch()` and combinators like `Promise.all`, and how `async/await` lets asynchronous code read almost like synchronous code while still never blocking the browser's main thread. It also covers the Fetch API in depth, including the easy-to-miss detail that `fetch` does not reject on HTTP error statuses, and closes with patterns for running requests in parallel, cancelling stale requests, and structuring error handling around network calls.

1. From Callbacks to Promises

The original async pattern in JavaScript was passing a callback function to be invoked later, when an operation completed. Nesting multiple dependent async operations this way produces deeply indented 'callback hell' that's hard to read and even harder to handle errors in consistently (each callback needs its own error-handling branch). A Promise represents a value that will exist in the future, in one of three states — pending, fulfilled, or rejected — and lets you chain `.then()` for success and `.catch()` for failure in a flat, readable sequence instead of nested callbacks.

JAVASCRIPT

```
// Promise chain – flat, each step handles the previous step's result
fetchUser(id)
  .then((user) => fetchOrders(user.id))
  .then((orders) => renderOrders(orders))
  .catch((err) => console.error('Failed to load orders:', err));
```

2. `async/await`

`async/await` is syntax sugar over Promises that lets asynchronous code read top-to-bottom like synchronous code. A function marked `async` always returns a Promise (even if you write a plain return value inside it, it gets wrapped). Inside an `async` function, `await` pauses execution of that function — and only that function — until the awaited Promise settles, without blocking the browser's main thread or any other code running elsewhere. Error handling reverts to familiar `try/catch` around the awaited calls, which most developers find more natural than chained `.catch()` calls once multiple steps are involved.

JAVASCRIPT

```
async function loadUserOrders(id) {
  try {
    const user = await fetchUser(id);
    const orders = await fetchOrders(user.id);
    renderOrders(orders);
  } catch (err) {
    console.error('Failed to load orders:', err);
  }
}
```

3. The Fetch API and the response.ok Trap

`fetch(url)` returns a Promise that resolves with a Response object as soon as the server responds with any status code at all — `fetch` only rejects on network-level failures (DNS failure, no connection, CORS block), never because the server returned a 404 or 500. This means checking `response.ok` (true for status codes 200-299) or `response.status` explicitly is mandatory; skipping it means a 404 page's error HTML gets silently parsed as if it were successful data. Parsing the body is itself async — `response.json()`, `response.text()`, and `response.blob()` all return Promises that must be awaited.

JAVASCRIPT

```
async function getUser(id) {
  const response = await fetch(`/api/users/${id}`);
  if (!response.ok) {
    throw new Error(`Request failed with status ${response.status}`);
  }
  return response.json();
}
```

4. Running Async Operations in Parallel

Awaiting independent requests one after another wastes time waiting sequentially for operations that don't depend on each other. `Promise.all([...])` runs multiple Promises concurrently and resolves once all of them succeed, rejecting immediately if any one of them rejects. `Promise.allSettled([...])` is the safer choice when you want results from every operation regardless of individual failures — it never rejects, and instead returns an array of `{status, value|reason}` objects for each input Promise.

JAVASCRIPT

```
// Sequential – slower, waits for each one before starting the next
const user = await fetchUser(id);
const posts = await fetchPosts(id);

// Parallel – both requests start immediately
const [user2, posts2] = await Promise.all([fetchUser(id), fetchPosts(id)]);

// Parallel, tolerant of individual failures
const results = await Promise.allSettled([fetchUser(id), fetchPosts(id)]);
results.forEach((r) => {
  if (r.status === 'fulfilled') console.log(r.value);
  else console.warn('One request failed:', r.reason);
});
```

5. Cancelling Requests with AbortController

A fetch left running when the component or context that requested it no longer cares (a user typed a new search query, or navigated away) wastes bandwidth and can cause a 'stale response overwrites fresh state' bug if it resolves after a newer request. `AbortController` provides a signal that can be passed to fetch and triggered later to cancel the in-flight request, causing the fetch Promise to reject with an `AbortError` that should typically be ignored (it's an intentional cancellation, not a real failure).

JAVASCRIPT

```
let controller;

async function search(query) {
  controller?.abort(); // cancel any previous in-flight search
  controller = new AbortController();
  try {
    const res = await fetch(`/api/search?q=${query}`, { signal:
    controller.signal });
    return res.json();
  } catch (err) {
    if (err.name === 'AbortError') return; // expected – a newer search
    // added this one
    super.throw(err);
  }
}
```

6. The Async Pitfalls of Array Iteration Methods

`forEach` does not wait for the callback's returned Promise, and does not support `await` meaningfully inside it — passing an async function to `forEach` fires off all the async calls without any guarantee about order or completion before `forEach` itself returns. `map` has the same issue but at least gives you back an array of Promises, which `Promise.all` can then be awaited on. When you genuinely need sequential async processing (one item must finish before the next starts), a plain `for...of` loop with `await` inside it is the clearest, most reliable choice.

JAVASCRIPT

```
// Broken: forEach doesn't wait – all requests fire, unordered, and errors are
// silently swallowed
items.forEach(async (item) => { await processItem(item); });

// Correct: sequential, ordered, errors propagate normally
for (const item of items) {
  await processItem(item);
}

// Correct: parallel, when order doesn't matter
await Promise.all(items.map((item) => processItem(item)));
```

7. Unhandled Rejections and Global Error Handling

A rejected Promise with no `.catch()` (or no surrounding `try/catch` if awaited) produces an unhandled promise rejection — logged as a console warning/error in browsers, and in Node.js capable of crashing the process depending on configuration. Listening for the global `unhandledrejection` event on window is a useful last line of defense for logging/monitoring in production, but it should never be relied on as the primary error-handling strategy; every async call site should have an explicit, deliberate error-handling path.

Common Pitfalls

- Forgetting that `fetch` only rejects on network failure, not on HTTP error statuses — always check `response.ok` before parsing the body.
- Passing an async function to `Array.prototype.forEach` and assuming it waits for each call to finish before moving to the next.
- Awaiting independent requests sequentially instead of using `Promise.all`, needlessly slowing down page load.
- Forgetting the `await` keyword and ending up with a Promise object instead of the resolved value, then trying to use it as if it were the data.
- Not handling `AbortError` separately from real errors, causing a canceled, intentional request to be logged/treated as a failure.
- Leaving stale in-flight requests uncanceled during rapid user input (search-as-you-type), letting an old response overwrite a newer one.
- Not wrapping `await` calls in `try/catch`, leaving unhandled promise rejections that silently fail in production.

Key Takeaways

- `async/await` is syntax over Promises, not a different concurrency model — a function marked `async` still returns a Promise.
- `await` pauses only the enclosing async function, never the browser's main thread.

- `fetch()` resolving successfully only means the network request completed — `response.ok` must be checked separately for HTTP-level success.
- `Promise.all` for concurrent success-required operations; `Promise.allSettled` when partial failure is acceptable.
- `AbortController` is the standard mechanism for cancelling stale in-flight requests, essential for search-as-you-type and rapid navigation.

Learn More

MDN — Using the Fetch API

https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch

MDN — Using Promises

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Using_promises

javascript.info — Async/await

<https://javascript.info/async-await>